

4. Architectural Design

(3 hours, 4 marks)

By Bidur Sapkota

4.1 Introduction and Importance

What is architectural design? [2 marks] (2082 Baishakh - IOE - Old Syllabus Relevant)

Why is architectural design important in software engineering? [2 marks] (2076 Ashwin - IOE - Old Syllabus Relevant)

Explain why it may be necessary to design the system architecture before specifications are written. [3 marks] (2075 Ashwin - IOE - Old Syllabus Relevant)

4.1 Introduction and Importance

What Is Architectural Design?

Architectural design is the process of defining the overall structure of a software system, including its major components, the relationships and interactions among those components, and the properties that govern their composition. It represents the "big picture" of the software before detailed component-level design begins. The output is an architectural model that depicts the system's structural elements (modules, objects, subsystems), their interfaces, the connectors that enable communication and coordination among them, and the constraints that define how they can be integrated.

4.1 Introduction and Importance

Software architecture is analogous to a building's floor plan. It shows the layout of rooms (components), how they connect (interfaces and connectors), and the overall structural form, without specifying plumbing or wiring details (component internals).

Why Is Architecture Important?

- **Facilitates communication among stakeholders.** The architecture serves as a common vocabulary for developers, managers, customers, and testers to discuss and reason about the system at a high level.

4.1 Introduction and Importance

Why Is Architecture Important?

- **Highlights early design decisions.** Architectural choices (e.g., selecting a layered vs. client-server style) have a profound impact on all subsequent development, including data structures, interfaces, control flow, testing strategy, and maintainability. Correcting architectural mistakes late in the project is extremely expensive.
- **Provides a transferable model.** Architectural styles and patterns are reusable across projects. Once an architecture is proven for one system, it can serve as a template for similar systems.

4.1 Introduction and Importance

Why Is Architecture Important?

- **Enables quality assessment before implementation.** The architecture can be analyzed for performance, reliability, security, and modifiability before any code is written.
- **Reduces risk.** By evaluating architectural alternatives early, the team can identify and mitigate structural risks before they propagate into the codebase.

4.1 Introduction and Importance

Why Design Architecture Before Detailed Specifications?

- The architecture defines the structural "skeleton" on which all detailed specifications and components will be built. Without it, specifications may describe features that are structurally incompatible or impossible to integrate.
- Early architectural design enables the team to identify subsystems and allocate work to parallel development teams.
- It allows assessment of non-functional requirements (performance, scalability, security) at a stage when changes are still inexpensive.
- In iterative/agile projects, a preliminary architecture (a "walking skeleton") guides sprint planning even as detailed requirements continue to evolve.

4.2 Architectural Design Principles

Architectural design is guided by a set of fundamental principles that apply regardless of the specific method, style, or programming language used:

Principle 1: Design should be traceable to the requirements model. Every element in the architectural design, such as subsystems, components, and interfaces, should map back to one or more requirements. If a component cannot be traced to a requirement, it may be unnecessary; if a requirement has no corresponding component, the design is incomplete.

4.2 Architectural Design Principles

Principle 2: Always consider the architecture first. Architecture affects interfaces, data structures, control flow, testability, and maintainability. Component-level details should only be addressed after the architecture is established.

Principle 3: Data design is as important as processing design. The way data objects are structured and stored fundamentally shapes the program's architecture and flow. A well-designed data architecture simplifies component implementation and improves processing efficiency.

4.2 Architectural Design Principles

Principle 4: Interfaces must be designed with care. Both internal interfaces (between components) and external interfaces (to users, devices, other systems) critically affect integration ease, error propagation, and design simplicity. A well-designed interface makes integration easier and helps testers validate component functions.

Principle 5: User interface design should prioritize ease of use. No matter how sophisticated the internal architecture, a poorly designed UI leads to the perception that the software is "bad."

Principle 6: Components should be functionally independent. Each component should be cohesive, meaning it is focused on a single, well-defined function. This simplifies development, testing, and maintenance.

4.2 Architectural Design Principles

Principle 7: Components should be loosely coupled. Minimize dependencies between components. High coupling increases error propagation and reduces maintainability.

Principle 8: Design representations should be easily understandable. The architectural model must serve as an effective communication medium for coders, testers, and maintainers.

Principle 9: Design should be developed iteratively. Each iteration refines the architecture, corrects errors, and strives for greater simplicity.

4.2 Architectural Design Principles

Principle 10: Agile projects still need architectural design. Even in agile development, a design model (kept lean and in sync with code) provides essential context that source code alone cannot convey, especially regarding high-level purpose and inter-module interactions.

4.3 Taxonomy of Architectural Styles

Explain client-server architecture with a suitable example. [5 marks] (2082 Baishakh - IOE - Old Syllabus Relevant)

Describe layered architecture for software with an example. [4 marks] (2082 Bhadra - IOE - Old Syllabus Relevant)

Differentiate repository model and layered model with their advantages and disadvantages. [3+3 marks] (2080 Bhadra - IOE - Old Syllabus Relevant)

Explain multi-tier (2-tier and 3-tier) architecture with examples. [4 marks] (2078 Bhadra - IOE - Old Syllabus Relevant)

4.3 Taxonomy of Architectural Styles

An architectural style defines a family of systems in terms of a pattern of structural organization. Each style describes:

- (1) a set of components that perform system functions,
- (2) a set of connectors that enable communication, coordination, and cooperation among components,
- (3) constraints that define how components can be integrated, and
- (4) semantic models that help understand the overall system properties.

4.3 Taxonomy of Architectural Styles

Data-Centered (Repository) Architecture

A central data store (database, file, or shared repository) resides at the center of the architecture. All other components access, add, update, or delete data from this store. Components operate independently and do not communicate directly with each other. Instead, they interact only through the shared data store.

- In a passive repository, client components access data independently of other clients. The repository simply stores and retrieves data on request.
- In a blackboard variant, the repository is active. It sends notifications to client components when data of interest changes, triggering further processing. This is useful for problems with no deterministic solution path (e.g., speech recognition, AI systems).

4.3 Taxonomy of Architectural Styles

Advantages:

Components can be changed or added independently without affecting other components (high integrability).

Centralized data management simplifies backup, security, and consistency.

Efficient for systems where many components share large volumes of data.

Disadvantages:

The repository is a single point of failure.

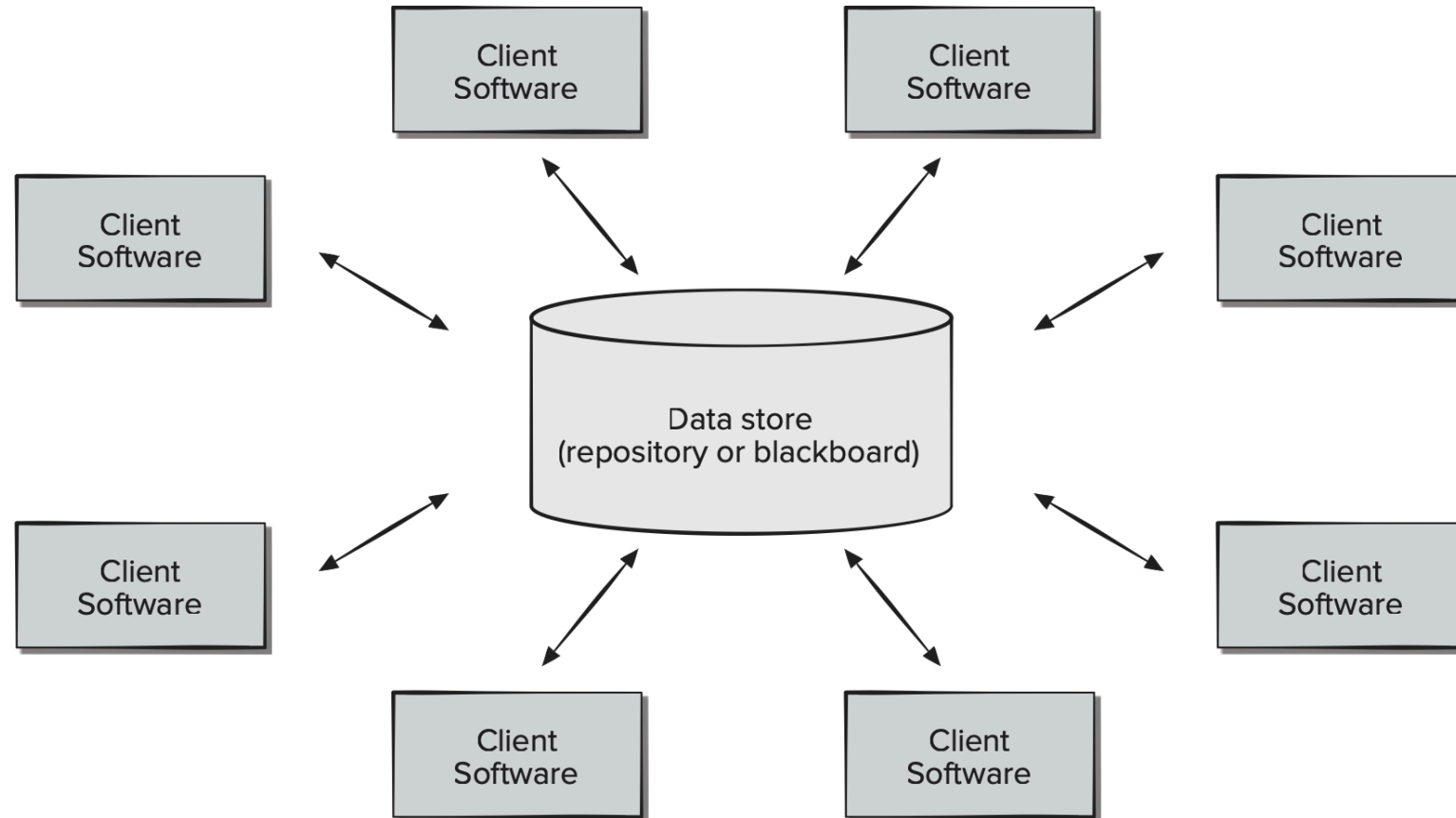
Components become tightly coupled to the data model, and changes to the data schema may require updates across many components.

It can also be difficult to distribute across multiple machines.

4.3 Taxonomy of Architectural Styles

Example: A CASE (Computer-Aided Software Engineering) tool suite where all tools (editor, compiler, debugger, version control) access a shared project repository.

**Data-centered
architecture**



4.3 Taxonomy of Architectural Styles

Data-Flow (Pipe-and-Filter) Architecture

Input data is transformed through a series of processing components (called filters) connected by pipes that transmit data from one filter to the next. Each filter works independently. It receives input in a defined format, processes it, and produces output in a defined format. Filters do not need to know the internal workings of neighboring filters.

4.3 Taxonomy of Architectural Styles

Advantages:

Supports reuse (filters can be recombined).

Easy to understand, add new filters, or replace existing ones.

Naturally supports concurrent processing.

Disadvantages:

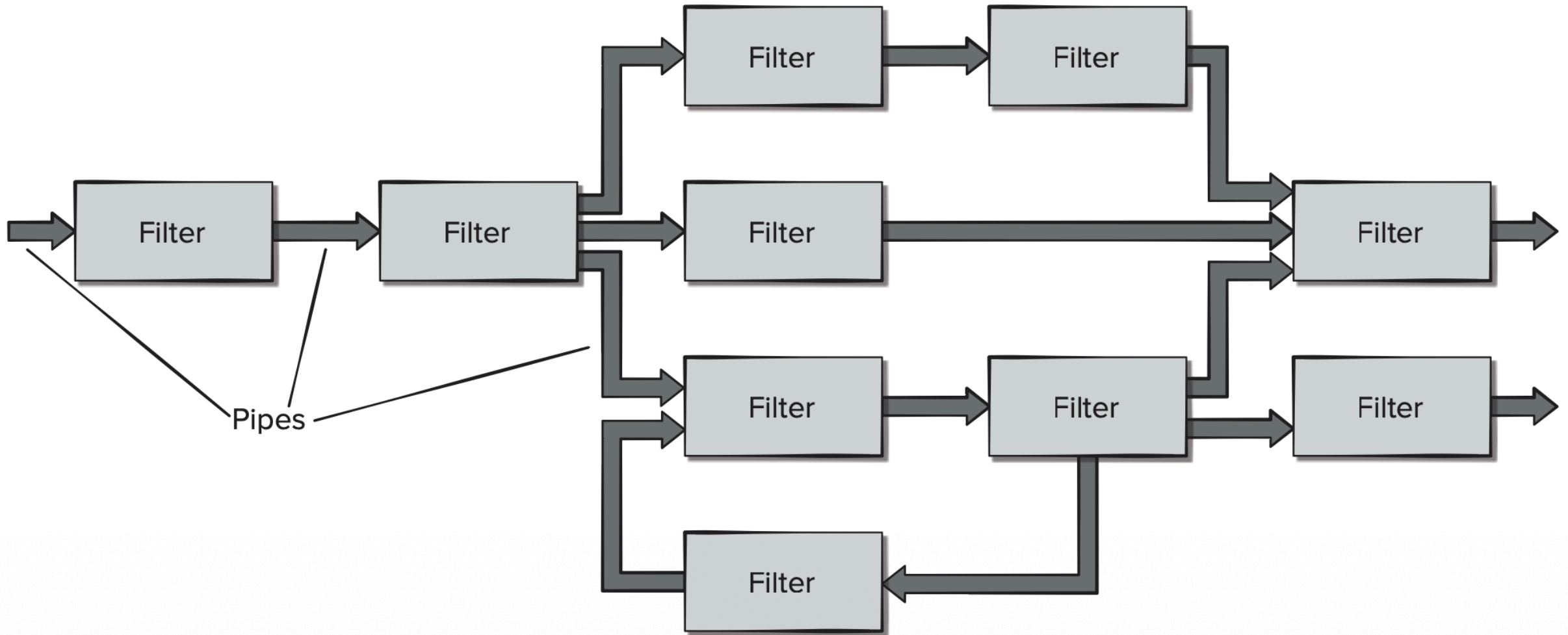
Not suitable for interactive applications.

Can have performance overhead due to data transformation between filters.

Not well-suited for complex, branching control flows.

Example: A Unix command pipeline: `cat file.txt | grep "error" | sort | uniq -c`. In this pipeline, each command is a filter connected by pipes.

4.3 Taxonomy of Architectural Styles



4.3 Taxonomy of Architectural Styles

Call-and-Return Architecture

A program structure where a "main" program invokes subprograms, which may in turn invoke further subprograms, forming a control hierarchy. Two common substyles:

- **Main program/subprogram:** Classic hierarchical decomposition where the main program delegates tasks to subprograms.
- **Remote procedure call (RPC):** The same hierarchy, but components are distributed across multiple computers on a network.

4.3 Taxonomy of Architectural Styles

Advantages:

Relatively easy to modify and scale.

Well-understood, traditional approach.

Clear control flow.

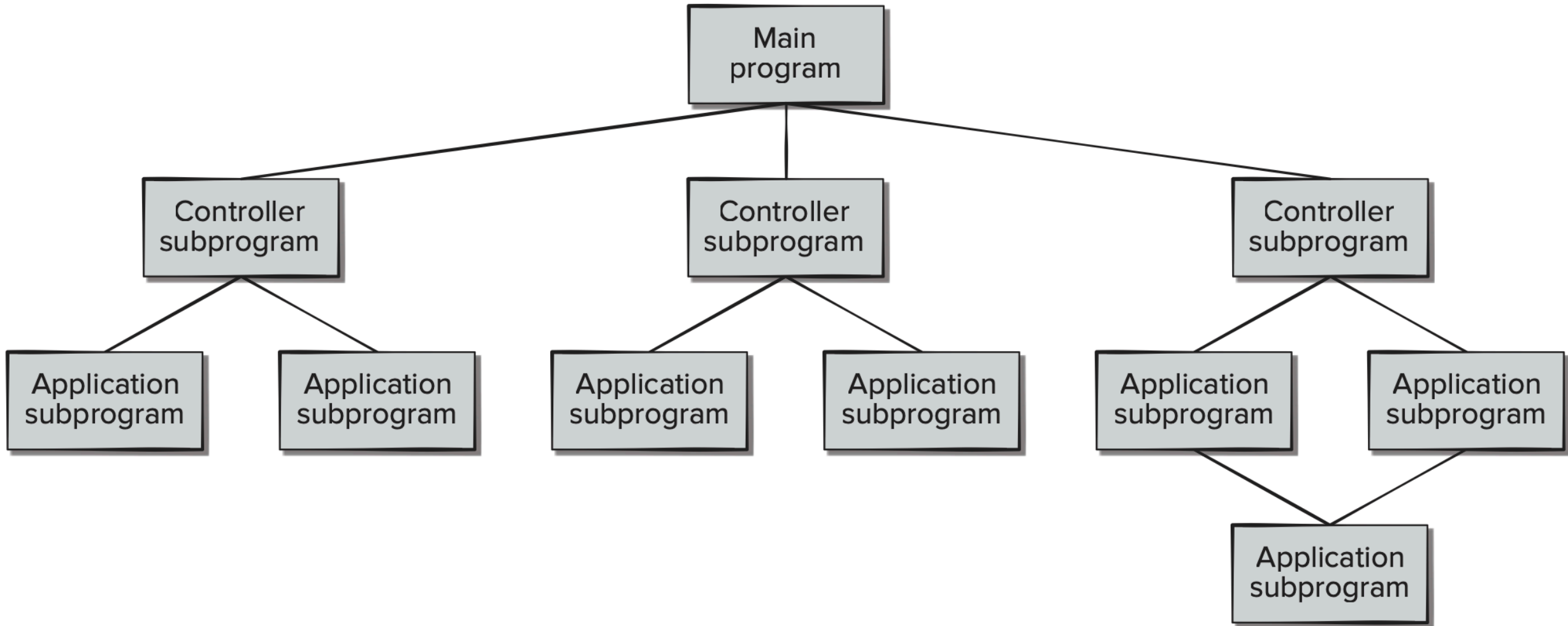
Disadvantages:

Can be rigid.

Changes to the hierarchy may require restructuring.

Tightly coupled control flow can reduce flexibility.

4.3 Taxonomy of Architectural Styles



4.3 Taxonomy of Architectural Styles

Object-Oriented Architecture

Components encapsulate both data and the operations that manipulate it.

Communication between components is achieved through message passing. Each object manages its own state and exposes behavior through well-defined interfaces.

Advantages:

Promotes reuse, modularity, and information hiding.

Changes to one object's internal representation do not affect others.

Natural mapping to real-world problem domains.

4.3 Taxonomy of Architectural Styles

Object-Oriented Architecture

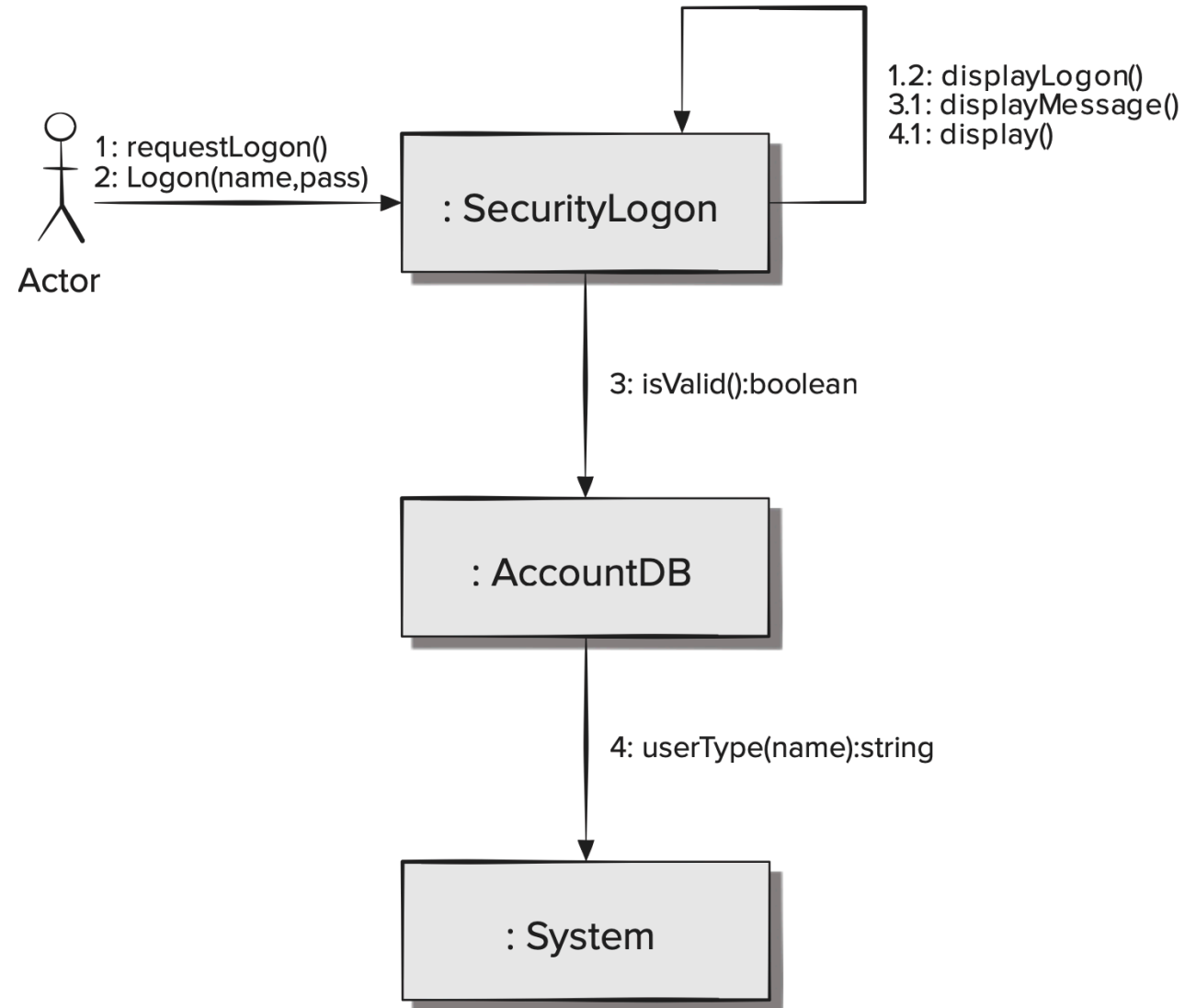
Disadvantages:

Can be harder to visualize the overall system structure.

Performance overhead from message passing.

Potential for complex interdependencies if not designed carefully.

4.3 Taxonomy of Architectural Styles



4.3 Taxonomy of Architectural Styles

Layered Architecture

The system is organized into horizontal layers, each providing a set of services to the layer above it and using services from the layer below. Each layer accomplishes operations that progressively become closer to the hardware level.

- The outermost layer handles user interface operations.
- The intermediate layers handle application logic and utility services.
- The innermost layer handles operating system interfacing and low-level services.

4.3 Taxonomy of Architectural Styles

Advantages:

Separation of concerns ensures that each layer has a clearly defined responsibility.

Layers can be developed, tested, and modified independently.

Changes in one layer generally do not affect others (as long as the interface is preserved).

Supports reuse of layers across applications.

4.3 Taxonomy of Architectural Styles

Disadvantages:

Performance overhead is a concern because requests may have to traverse multiple layers.

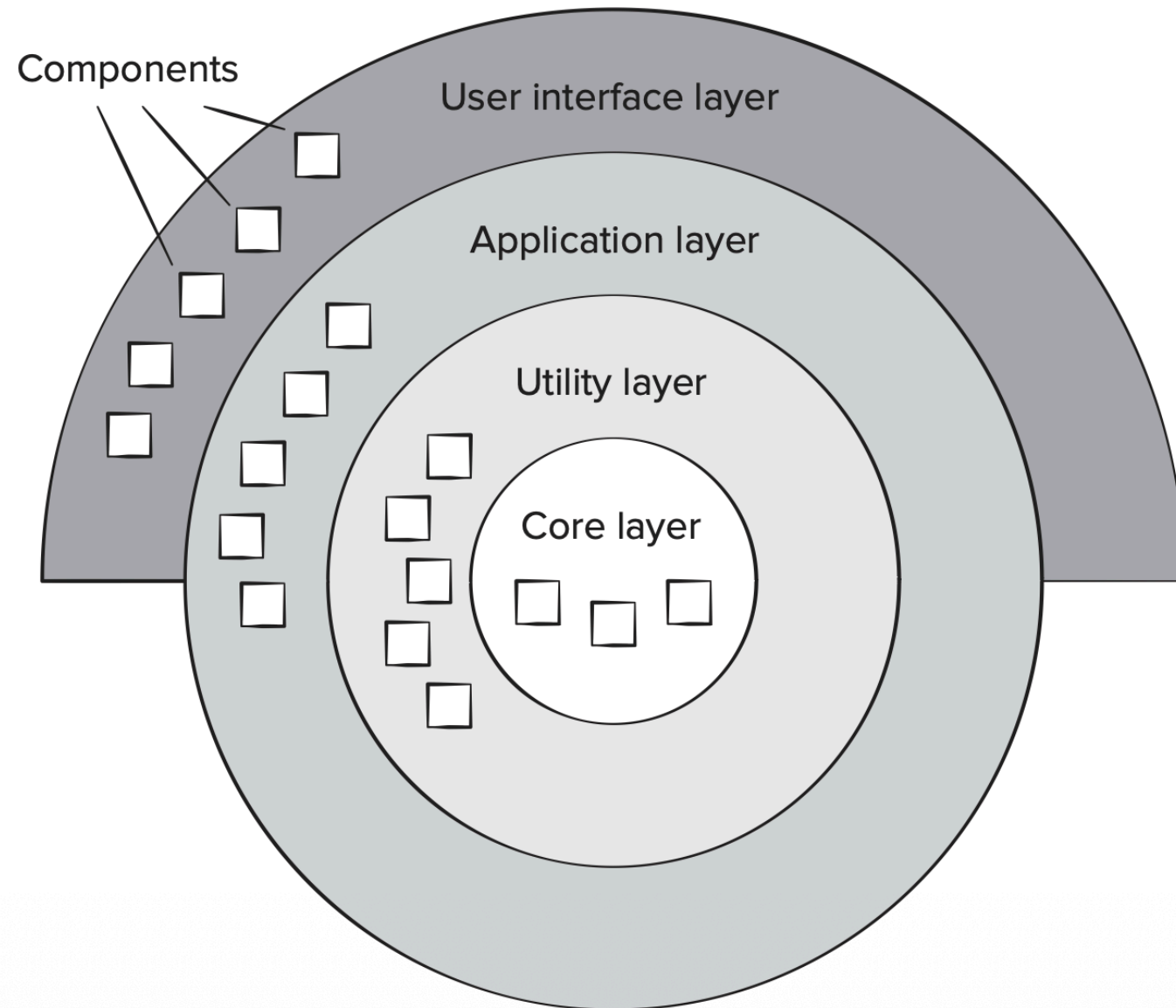
Not all systems decompose cleanly into layers. Excessive layering can introduce complexity and rigidity.

Example: An Operating System architecture:

- (1) User Interface layer (Shell or GUI),
- (2) System Utilities layer (File management and process tracking),
- (3) Kernel layer (Core memory management and CPU scheduling), and
- (4) Hardware Abstraction layer (Device drivers communicating with physical hardware).

4.3 Taxonomy of Architectural Styles

Layered architecture



4.3 Taxonomy of Architectural Styles

Client-Server Architecture

A distributed architecture that partitions work between servers (providers of resources/services) and clients (requesters of services). The server hosts shared resources (data, processing logic, files) and listens for client requests. Clients present the user interface and send requests to the server over a network.

Advantages:

Centralized management of data and security.

The architecture is scalable because servers can be upgraded or additional servers can be added.

4.3 Taxonomy of Architectural Styles

Advantages:

It is resource-efficient since thin clients can access powerful server-side processing.
Data integrity is maintained at the server.

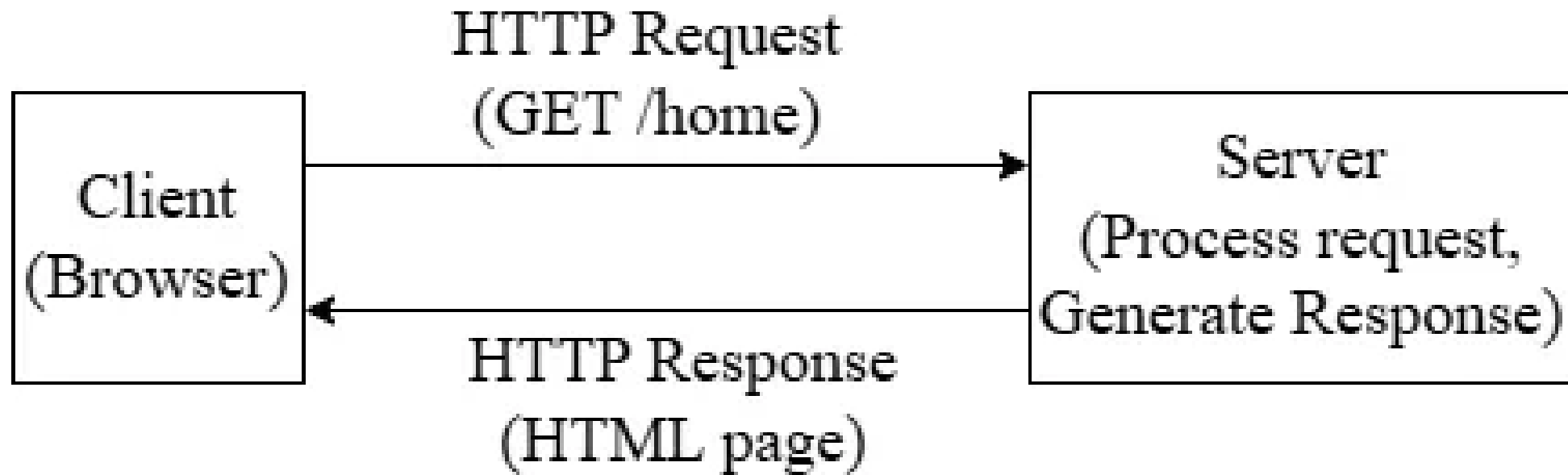
Disadvantages:

Server is a single point of failure.
Entirely dependent on network connectivity.
Server can become a bottleneck under heavy load.
Higher infrastructure cost.

4.3 Taxonomy of Architectural Styles

Example: In an online banking system, the mobile app (client) sends transaction requests to the bank's application server, which processes business logic and communicates with the database server to fetch or update account information. The server returns results to the client for display.

4.3 Taxonomy of Architectural Styles



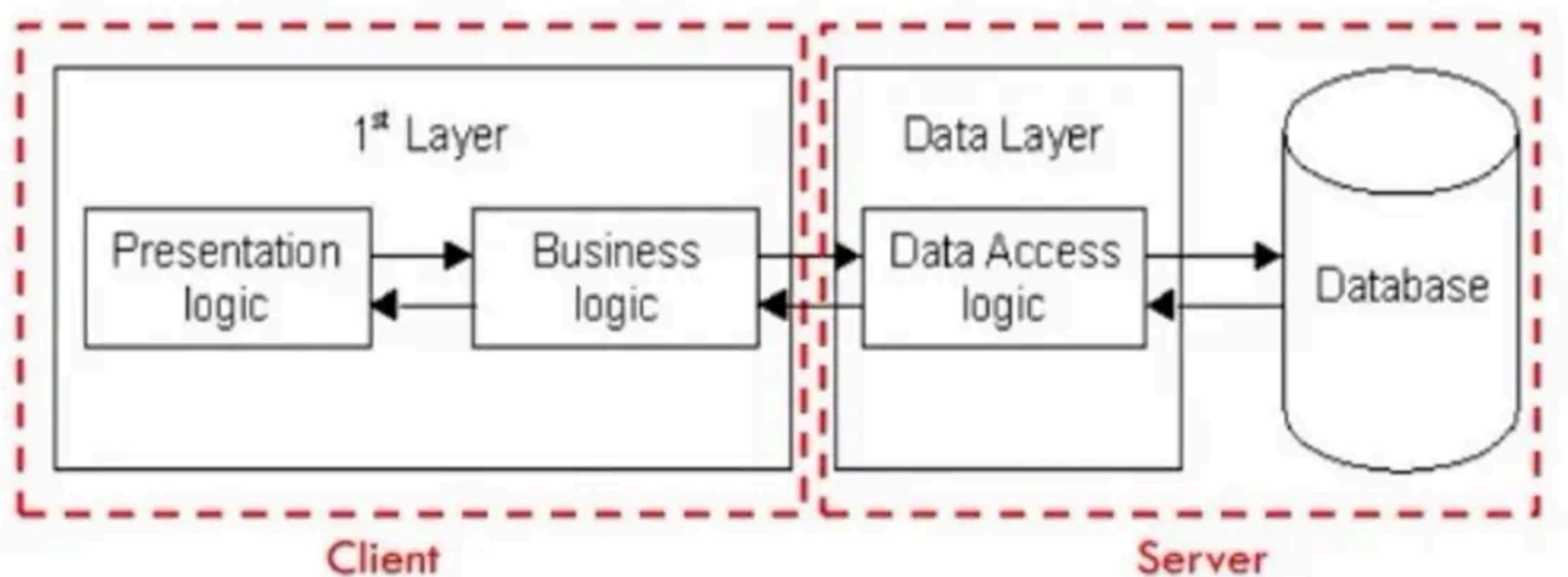
4.3 Taxonomy of Architectural Styles

Multi-Tier Architecture

An extension of client-server that separates functionality into distinct tiers, each running on separate infrastructure:

2-Tier Architecture: The client communicates directly with the server. The client handles the UI and some application logic; the server handles data storage and remaining logic. Example: A desktop database application where the client application connects directly to a database server.

4.3 Taxonomy of Architectural Styles



4.3 Taxonomy of Architectural Styles

3-Tier Architecture: This style adds a middle tier between client and server. The three tiers are:

- (1) the Presentation tier, which is the client UI,
- (2) the Application/Logic tier, which handles business rules and processing, and
- (3) the Data tier, which is the server database.

Each tier can be developed, deployed, and scaled independently.

Example: In an e-commerce website, the browser (presentation tier) sends requests to a web/application server (logic tier) that processes orders, applies business rules, and queries the database server (data tier).

4.3 Taxonomy of Architectural Styles

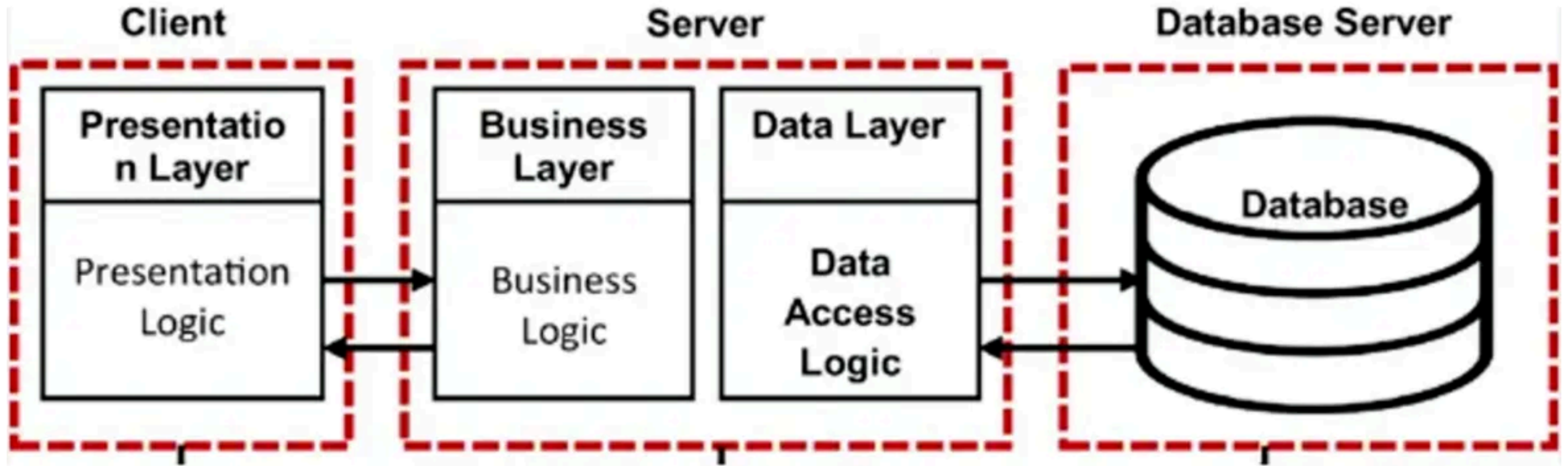


Fig: Three-tier Client-Server Architecture

4.3 Taxonomy of Architectural Styles

Advantages of 3-tier over 2-tier:

Better separation of concerns.

The middle tier can be scaled independently.

Security is improved because the database is not directly exposed to clients.

Maintenance is easier because business logic changes do not affect the client or database.

4.3 Taxonomy of Architectural Styles

Model-View-Controller (MVC) Architecture

A widely used architectural pattern, particularly in web and mobile applications, that separates an application into three interconnected components:

- The Model encapsulates application data, business logic, and rules. It manages data access and state.
- The View presents data from the model to the user. It handles all UI rendering and display logic.
- The Controller receives user input, interprets it, and invokes appropriate model operations. It selects the view to display the response.

4.3 Taxonomy of Architectural Styles

The user interacts with the View. The Controller receives the request and invokes the Model. The Model processes the data and returns the results to the Controller. The Controller then selects the appropriate View, and the View renders the response to the user.

Advantages:

- Clear separation of concerns allows designers to work on the UI (View) independently of developers working on business logic (Model).
- Supports multiple views for the same data.
- Facilitates testing (Model can be tested independently).

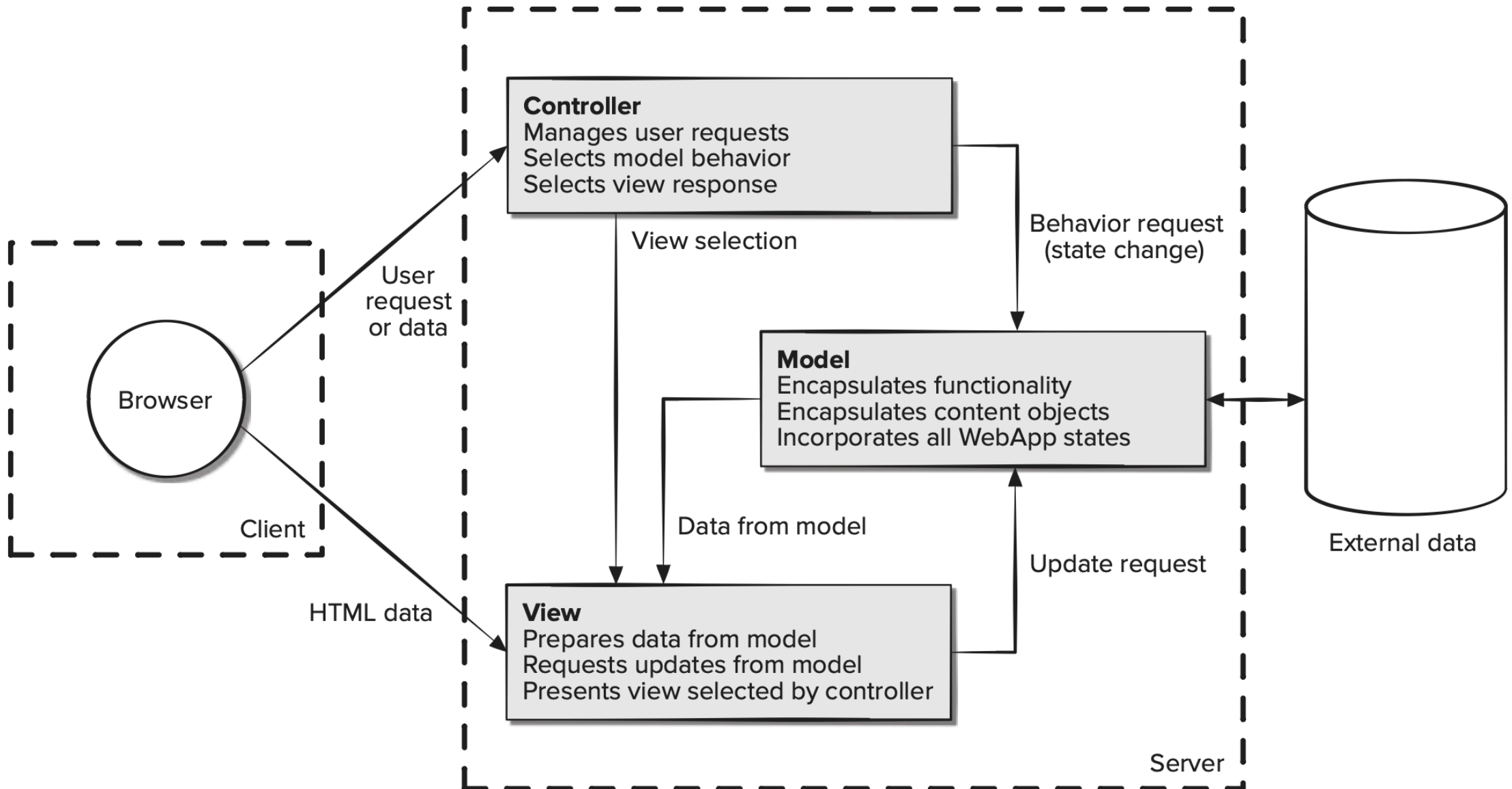
4.3 Taxonomy of Architectural Styles

Disadvantages:

Can increase complexity for simple applications.

Tight coupling between Controller and Model can emerge if not designed carefully.

4.3 Taxonomy of Architectural Styles



4.3 Taxonomy of Architectural Styles

Repository Model	Layered Model
A central data store is accessed by independent components.	The system is organized into horizontal layers, each serving the layer above it.
All components interact through the shared data store.	Each layer communicates only with adjacent layers.
It is efficient for systems with large shared data sets.	Data passes through multiple layers, which may introduce overhead.
Components are coupled to the data model, not to each other.	Layers are coupled only through well-defined interfaces.

4.3 Taxonomy of Architectural Styles

Repository Model	Layered Model
Adding or removing components is easy, but changing the data schema is hard.	Changing a layer is easy as long as the interface is preserved.
The data store is a single point of failure.	There is no single point of failure unless a critical layer fails.
It is best suited for systems with shared, complex data such as CASE tools, IDEs, and data warehouses.	It is best suited for systems with clear functional separation such as web applications, OS, and network protocols.

4.4 Modular Design, Cohesion, and Coupling

What are the different modular decomposition styles used during system design? Explain with examples. [5 marks] (2075 Chaitra - IOE - Old Syllabus Relevant)

How is the modular decomposition concept practiced in the system design process? [4 marks] (2071 Shrawan - IOE - Old Syllabus Relevant)

4.4 Modular Design, Cohesion, and Coupling

Modular Design

Modularity is the most common manifestation of the "separation of concerns" principle. The software is divided into separately named and addressable components called modules, each of which is integrated to satisfy overall system requirements. A monolithic program (one large module) is nearly impossible to understand, test, or maintain. Modularization makes software intellectually manageable.

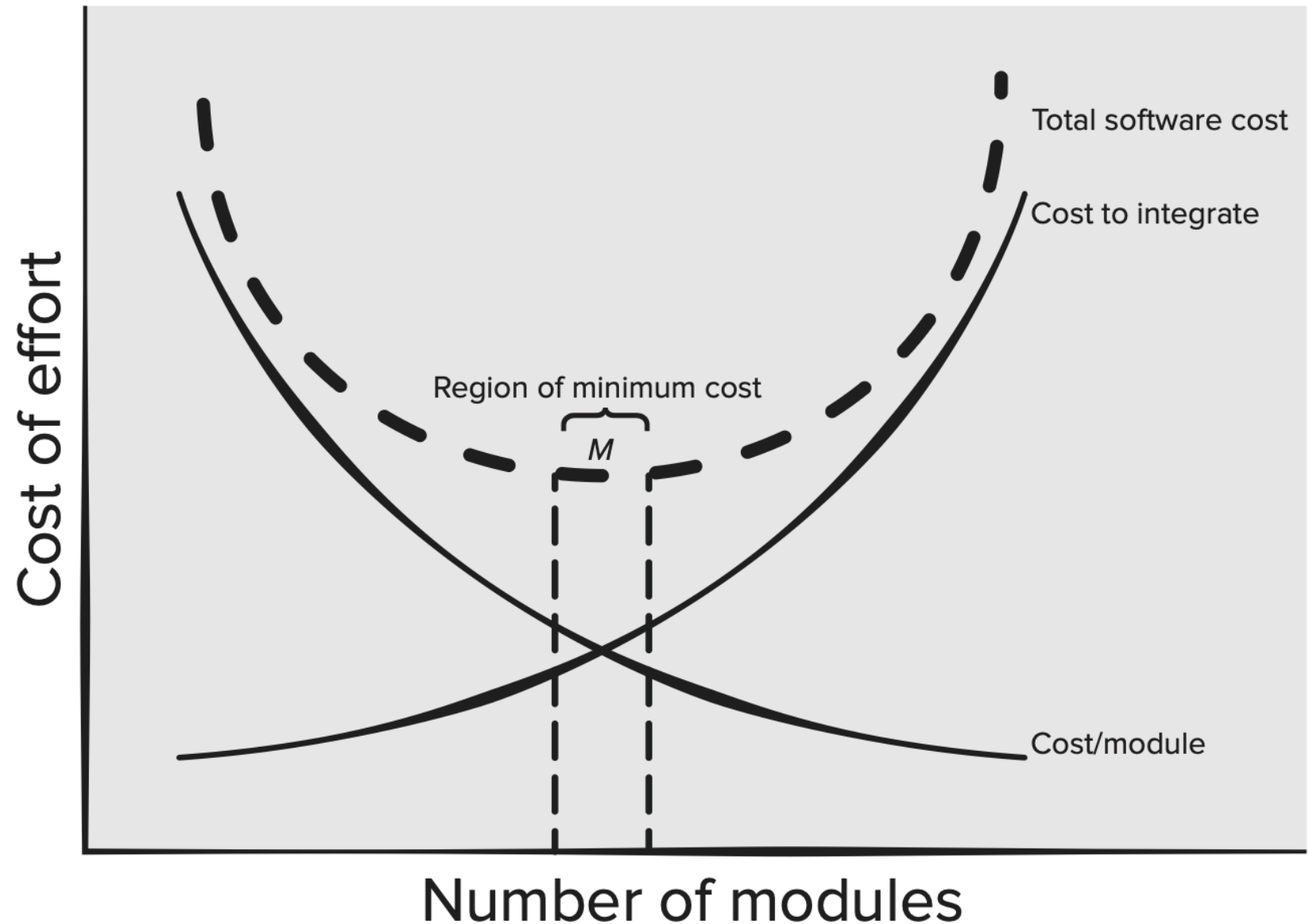
4.4 Modular Design, Cohesion, and Coupling

Why modularize? Development can be more easily planned. Software increments can be defined and delivered. Changes can be more easily accommodated. Testing, debugging, and long-term maintenance can be conducted more efficiently and without serious side effects.

The modularization trade-off: As the number of modules increases, the cost per individual module decreases (smaller, simpler modules). However, the cost of integrating modules increases (more interfaces, more communication). There is an optimal number of modules (M) that minimizes total development cost. Too few modules means each is too complex; too many modules means excessive integration overhead.

4.4 Modular Design, Cohesion, and Coupling

**Modularity
and software
cost**



4.4 Modular Design, Cohesion, and Coupling

Modular Decomposition Styles

Functional decomposition (top-down): The system is decomposed by function. A high-level function is broken into sub-functions, which are further broken into smaller sub-functions until each module performs a single, well-defined task.
Example: An order processing system decomposed into modules for "Validate Order," "Calculate Total," "Process Payment," and "Generate Invoice."

4.4 Modular Design, Cohesion, and Coupling

Modular Decomposition Styles

Object-oriented decomposition: The system is decomposed into objects (classes) that encapsulate data and the operations on that data. Modules correspond to real-world entities. Example: An e-commerce system decomposed into objects like `Customer` , `Product` , `ShoppingCart` , `Order` , and `Payment` .

Information Hiding

The principle of information hiding (proposed by David Parnas) states that modules should be designed so that internal details (algorithms, data structures, design decisions) are hidden from other modules. Only the module's interface (what it does, not how it does it) is visible to the outside.

- Effective modularity is achieved by defining independent modules that communicate only information necessary to achieve software function.
- Information hiding reduces error propagation. Inadvertent errors introduced during modification are less likely to spread to other parts of the software.
- It provides the greatest benefits during testing and maintenance, when modifications are frequent.

4.4 Modular Design, Cohesion, and Coupling

Functional Independence

A module is functionally independent if it performs a single, well-defined function and interacts with other modules only through simple, well-defined interfaces.

Functional independence is a direct outgrowth of separation of concerns, modularity, abstraction, and information hiding.

Importance:

- Independent modules are easier to develop (function is compartmentalized, interfaces are simplified).
- Independent modules are easier to test and maintain (secondary effects of modifications are limited, error propagation is reduced).

4.4 Modular Design, Cohesion, and Coupling

- Independent modules are more reusable.

Functional independence is assessed using two qualitative criteria: cohesion and coupling. The goal is high cohesion and low coupling.

4.4 Modular Design, Cohesion, and Coupling

Cohesion

Cohesion measures how closely related and focused the responsibilities within a single module are. A cohesive module performs a single task, requiring little interaction with other modules. Types of cohesion (from worst to best):

Coincidental cohesion means that module parts are completely unrelated and grouped arbitrarily. This is the weakest form. Example: A "utility" module containing unrelated functions like `printReport()`, `calculateTax()`, and `validateEmail()` simply because they were written by the same developer.

4.4 Modular Design, Cohesion, and Coupling

Logical cohesion means that module parts perform logically similar functions but are otherwise unrelated. Example: A module that handles all input operations (keyboard, file, network) through a single function with a control flag.

Temporal cohesion means that module parts are grouped because they are executed at the same time (e.g., initialization, cleanup). Example: A `startup()` function that initializes the database connection, loads configuration, and sets the system clock.

Procedural cohesion means that module parts are grouped because they always execute in a specific sequence. Example: A module that reads a record, validates it, and writes it to a file. The steps are related only by sequence, not by function.

4.4 Modular Design, Cohesion, and Coupling

Communicational cohesion means that module parts operate on the same data or contribute to the same output. Example: A module that generates a report by reading customer data, calculating totals, and formatting the output. All operations share the same data.

Sequential cohesion means that the output of one part serves as the input for the next part. Example: A module where `parseInput()` produces data consumed by `validateData()`, whose output is consumed by `transformData()`.

4.4 Modular Design, Cohesion, and Coupling

Functional cohesion means that every element of the module is essential for the execution of a single, well-defined function. This is the strongest and most desirable form. Example: A `computeSquareRoot()` module that performs only the square root computation and returns the result.

Always strive for functional cohesion. Communicational and sequential cohesion are also acceptable. Avoid coincidental, logical, and temporal cohesion.

4.4 Modular Design, Cohesion, and Coupling

Coupling

Coupling measures the degree of interdependence between modules. Low coupling means modules are relatively independent; high coupling means modules are heavily dependent on each other. Types of coupling (from worst to best):

Content coupling occurs when one module directly accesses or modifies the internal data or code of another module. This violates information hiding and is the most harmful form. Example: Module A directly modifies a local variable inside Module B.

4.4 Modular Design, Cohesion, and Coupling

Common coupling occurs when multiple modules share the same global data. A change to the global data structure can affect all modules that use it. Example: Several modules read and write a global configuration array.

Control coupling occurs when one module passes a control flag to another module that directs its internal logic. Example: Module A calls `processRecord(flag)` where the flag tells Module B whether to insert, update, or delete. Changes to Module B's logic may require changes to the flag's meaning in Module A.

4.4 Modular Design, Cohesion, and Coupling

Stamp coupling occurs when modules share a composite data structure (e.g., a struct or object) but each uses only part of it. The modules become dependent on the structure's format. Example: Passing an entire `Customer` object to a module that only needs the customer's email address.

Data coupling occurs when modules communicate only through simple parameters (primitive data types). Each parameter is an independent data item needed by the module. This is the most desirable form. Example: `calculateArea(length, width)` where only the required primitive values are passed.

Always strive for data coupling. Avoid content coupling and common coupling. Minimize control coupling by redesigning interfaces.

4.4 Modular Design, Cohesion, and Coupling

Relationship Between Cohesion and Coupling

Cohesion and coupling are inversely related. Increasing cohesion within modules tends to decrease coupling between modules. A system with high cohesion and low coupling is easier to understand, test, maintain, and reuse. This is the hallmark of a well-designed modular system.